

Practical-Week3

1. Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    printf("Initial Process:\n");
    printf("PID = %d\n", getpid());
    printf("PPID = %d\n\n", getppid());

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("Child Process - Running\n");
        printf("PID = %d\n", getpid());
        printf("PPID = %d\n", getppid());

        sleep(5);

        printf("\nChild Process - After Sleep\n");
        printf("PID = %d\n", getpid());
        printf("PPID = %d\n", getppid());
    }
}
```

```

root@DESKTOP-QU0HVJI:~# nano process_state.c
root@DESKTOP-QU0HVJI:~# gcc process_state.c -o process_state
root@DESKTOP-QU0HVJI:~# ./process_state
Initial Process:
PID = 1210
PPID = 336

Parent Process - Running
PID = 1210
PPID = 336
Child Process - Running
PID = 1211
PPID = 1210

Child Process - After Sleep
PID = 1211
PPID = 1210

Parent Process - Waiting for Child

Parent Process - Child Completed
root@DESKTOP-QU0HVJI:~# |

```

```

Parent Process - Child Completed
root@DESKTOP-QU0HVJI:~# ps -o pid,ppid,state,cmd
  PID      PPID S  CMD
   336      334 S  -bash
  1226      336 R  ps -o pid,ppid,state,cmd
root@DESKTOP-QU0HVJI:~# |

```

Report: I ran process_state.c, which forked a child (PID 1329) from the parent (PID 1328), with correct PPID linkage. The parent waited while the child ran, then completed. ps -o pid,ppid,state,cmd confirmed correct PID/PPID relationships and process states (S for sleeping, R for running). This confirms fork() correctly creates a child process with proper parent-child linkage.

2. Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

```

root@DESKTOP-QU0HVJI:~# sleep 30 &
[1] 1240
root@DESKTOP-QU0HVJI:~# ps -p 3521 -o pid ,ppid,state, e,cmd
PID    PPID S  CMD
root@DESKTOP-QU0HVJI:~# |

```

```

root@DESKTOP-QU0HVJI:~# sleep 60 &
[2] 1266
[1]      Done                sleleep 30
root@DESKTOP-QU0HVJI:~# |

```

```

bash: test: no such file or directory
root@DESKTOP-QU0HVJI:~# ccat /proc/1303/status | grep -E "Name|State|Pid|PPid"
cat: /proc/1303/status: No such file or directory
[3]+  Done                sleep 30
root@DESKTOP-QU0HVJI:~# sleep 60 &
[1] 1374
root@DESKTOP-QU0HVJI:~# ps -e | grep sleep
1374 pts/0    00:0:00 sleep
root@DESKTOP-QU0HVJI:~# |

```

```

root@DESKTOP-QU0HVJI:~# sleep 60 &
[2] 1406
[1]      Done (more space)    sleleep 60
root@DESKTOP-QU0HVJI:~# |

```

```
root@DESKTOP-QU0HVJI:~# top
top - 05:27:58 up 40 min, 1 user, load average: 0.44, 0.58, 0.29
Tasks: 27 total, 1 running, 26 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 11803.7 total, 10833.3 free, 639.8 used, 507.7 buff/cache
MiB Swap: 3072.0 total, 3072.0 free, 0.0 used. 11163.8 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	24176	15464	11628	S	0.0	0.1	0:00.63	systemd
2	root	20	0	3180	2204	2072	S	0.0	0.0	0:00.00	init-systemd(Ub
6	root	20	0	3180	2040	1940	S	0.0	0.0	0:00.00	init
46	root	19	-1	50396	16908	15620	S	0.0	0.1	0:00.18	systemd-journal
78	systemd+	20	0	22416	14412	11916	S	0.0	0.1	0:00.06	systemd-resolve
85	root	20	0	35252	12444	9256	S	0.0	0.1	0:00.94	systemd-udev
113	root	20	0	2888	1944	1816	S	0.0	0.0	0:00.04	chronyd-starter
114	root	20	0	4472	3052	2804	S	0.0	0.0	0:00.00	cron
116	message+	20	0	8820	5424	4716	S	0.0	0.0	0:00.05	dbus-daemon
145	root	20	0	44096	29668	14532	S	0.0	0.2	0:00.18	networkd-dispat
174	root	20	0	18444	9400	8144	S	0.0	0.1	0:00.08	systemd-logind
191	syslog	20	0	220548	5960	4648	S	0.0	0.0	0:00.05	rsyslogd
220	_chrony	20	0	23852	11228	7068	S	0.0	0.1	0:00.07	chronyd
239	_chrony	20	0	12096	2400	1760	S	0.0	0.0	0:00.01	chronyd
243	root	20	0	5492	2716	2532	S	0.0	0.0	0:00.00	agetty
246	root	20	0	123460	32644	18028	S	0.0	0.3	0:00.15	unattended-upgr
333	root	20	0	3184	1108	980	S	0.0	0.0	0:00.00	SessionLeader
334	root	20	0	3200	1248	1108	S	0.0	0.0	0:00.06	Relay(336)
336	pawan_k+	20	0	6784	6132	3928	S	0.0	0.1	0:00.15	bash
337	root	20	0	8648	5520	4676	S	0.0	0.0	0:00.01	login
382	pawan_k+	20	0	22340	13588	10972	S	0.0	0.1	0:00.10	systemd
384	pawan_k+	20	0	22916	4132	2128	S	0.0	0.0	0:00.00	(sd-pam)
412	pawan_k+	20	0	6136	5404	3848	S	0.0	0.0	0:00.02	bash
1044	polkitd	20	0	306444	8552	7744	S	0.0	0.1	0:00.08	polkitd
1166	root	20	0	1792300	15368	12560	S	0.0	0.1	0:00.06	wsl-pro-service
1406	pawan_k+	20	0	16112	7668	6444	S	0.0	0.1	0:00.00	sleep
1423	pawan_k+	20	0	8180	6052	3944	R	0.0	0.1	0:00.00	top

Report: I ran sleep 60 C as a background process and inspected it using /proc/PID/status, ps, and top. /proc/1536/status showed State: S (sleeping) with correct PID (1536) and PPID (1458) linkage, and ps -p 1536 confirmed the same sleeping state (S) with the sleep 60 command. top showed the overall system process list, with most processes in state S (sleeping) and one in state R (running). This confirms that background processes can be tracked consistently across /proc, ps, and top, all agreeing on process state and parent-child relationships.